

AI AGENT BLUEPRINT

SmartShelf Vision

Retail Shelf Inventory & Out-of-Stock Detection

VisionLY

Presented By: Yilin Leng, Lufei Yu

ITAI 1378 Computer Vision in Artificial Intelligence

Professor Patricia McManus

Houston City College

Fall 2026

1. Problem Statement

Retail stores need to regularly inspect shelves to ensure products are in stock and adequately displayed. In traditional inventory processes, staff must manually patrol aisles, identify products, count visible items, and determine which ones require restocking. This process is repetitive and time-consuming, particularly given the large number of shelves and the wide variety of products found in stores. Manual counting can also lead to inconsistent results due to variations in staff methods or the likelihood of overlooking items that are partially obscured, stacked, or difficult to see clearly.

Another challenge is that traditional inventory systems may not accurately reflect the actual state of shelf displays. A product might appear as "in stock" in the records while the shelf itself is empty or nearly depleted, creating a discrepancy between recorded inventory and actual shelf availability. Failure to identify low-stock items promptly—even when the product is available elsewhere in the store—can result in customers facing empty shelves, thereby diminishing the shopping experience.

SmartShelf Vision aims to address this issue by using computer vision technology to automatically analyze photographs of retail shelves. Instead of requiring staff to manually count visible items, the system employs the YOLOv8 object detection model to locate products within the images. Application logic then tallies the detected items and compares the count against preset inventory thresholds. If the number of visible items falls below the threshold, the system flags the product as "low stock" or "out of stock."

This project focuses on detecting and counting items visible on the shelf surface rather than attempting a comprehensive inventory count for the entire store. This distinction is crucial, as some items may be obscured by other products or located outside the camera's field of view. By focusing on what is actually detectable within the images, SmartShelf Vision offers a practical and quantifiable computer vision use case suitable for a "Tier 1" project.

This project aims to transform simple shelf photos into valuable inventory information:

Shelf images → Product detection → Product counting → Inventory assessment → Restocking alerts

This approach demonstrates how computer vision can reduce repetitive manual labor, enable more consistent shelf monitoring, and provide store staff with a faster way to identify products requiring attention.

Business Goal

- Reduce the amount of manual shelf inspection required.
- Provide a consistent method for counting visible products.
- Identify potential low-stock or out-of-stock situations.
- Produce an easy-to-understand visual result for a store associate.

2. Chosen Option — Tier 1

The project uses Tier 1: one model doing one job.

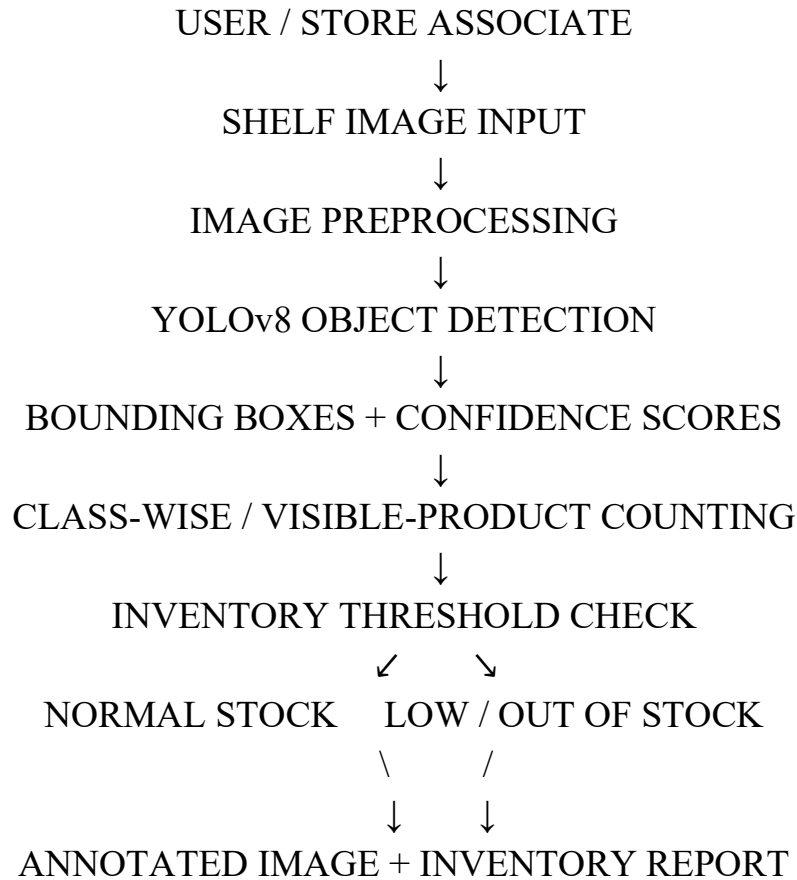
The model's primary job is object detection. Application logic outside the model handles counting and inventory-status decisions.

Component	Responsibility	Technology
YOLOv8	Detect visible product objects and return bounding boxes/confidence scores	Ultralytics / PyTorch
Counting Logic	Group detections and calculate product counts	Python / NumPy
Inventory Logic	Compare counts with a threshold and assign status	Python
Visualization	Draw boxes, labels, counts, and alerts	OpenCV

3. Agent Architecture & Diagram

The SmartShelf Vision agent is designed as a lightweight perception-and-decision workflow. A user provides a shelf image. The vision model detects visible products. The agent then uses deterministic application logic to count detections and evaluate inventory status.

Diagram:



Agent Inputs and Outputs

Input	Output
Shelf photograph	Annotated shelf photograph
Detection results	Product counts
Inventory threshold	Normal / Low / Out-of-stock status
Model confidence	Detection results for evaluation

4. Computer Vision Connection

Computer vision is the central capability of SmartShelf Vision. Object detection is used because the system must locate individual visible products in an image, not simply decide what category the entire image belongs to.

- Image input: a retail shelf image captured by phone or obtained from a public dataset.
- Object detection: YOLOv8 predicts bounding boxes, class information, and confidence scores.
- Post-processing: Python/OpenCV processes the detection results.
- Counting: detections are tallied by class or product group when class labels are available.
- Inventory evaluation: counts are compared with predefined thresholds.
- Visualization: OpenCV generates an annotated image showing the detected products and inventory status.

Important limitation: a camera can only count products that are sufficiently visible to the model. Products hidden behind other products may be missed. Therefore, the project emphasizes visible/front-facing product counting.

5. Agent Framework

The framework is intentionally simple so that the team can demonstrate the AI-agent workflow without adding unnecessary models or complex orchestration.

Layer	Role	Implementation
Perception	Understand the shelf image	YOLOv8
Processing	Clean and organize detection results	Python / NumPy
Decision	Evaluate inventory condition	Threshold-based Python logic
Action / Output	Communicate the result	OpenCV annotation + report

Because this is Tier 1, the agent does not require a separate large language model, multiple AI models, or autonomous external actions. The 'agent' behavior is represented by the coordinated sequence from perception to interpretation and action/output.

6. Tools & Data Sources

Tool / Source	Purpose	Planned Use
YOLOv8	Object detection	Pretrained YOLOv8 Small baseline
PyTorch	Deep learning framework	Model execution/inference
OpenCV	Computer vision and visualization	Preprocessing and annotated outputs
NumPy	Numerical processing	Detection/count processing
Google Colab	Development environment	Model testing and experiments
GitHub	Version control/documentation	Code, README, project tracking
SKU-110K / RP2K	Retail shelf image datasets	Candidate training/testing data
Roboflow datasets	Annotated computer-vision data	Alternative dataset source
Smartphone photos	Real-world test images	Optional custom evaluation images

Data-source selection will be finalized before evaluation. Dataset licenses and usage requirements should be checked before redistributing images or annotations in the GitHub repository.

7. Build Plan

Phase 1 — Blueprint: Finalize scope, repository structure, dataset plan, baseline configuration, and success metrics.

Phase 2 — First Demo: Load YOLOv8, run inference on sample shelf images, and save annotated results.

Phase 3 — Counting Logic: Implement class-wise or product-group counting and filtering of low-confidence detections.

Phase 4 — Inventory Logic: Create configurable thresholds for Normal, Low Stock, and Out-of-Stock states.

Phase 5 — Evaluation: Measure mAP, precision, recall, count accuracy, count error, and inference time.

Phase 6 — Packaging: Organize source code, examples, README, results, presentation, and demonstration video.

Proposed Repository Structure

SmartShelf-Vision/

```
|— README.md
|— data/README.md
|— docs/proposal.pdf
```

8. Expected Changes / Improvements

The expected changes are measured against the current manual or baseline workflow. The project should improve consistency and speed of visible-product counting while producing a clearer visual inventory result.

- From manual visual inspection to automated image-based detection.
- From manual counting to programmatic product counting.
- From unstructured observations to standardized inventory status.
- From raw shelf images to annotated images with bounding boxes and counts.
- From delayed identification to immediate low-stock/out-of-stock indication after inference.
- From a generic pretrained detector to a documented retail-specific workflow and evaluation process.

Target Performance

Metric	Target	Purpose
mAP@0.5	≥ 0.88	Measure detection quality
Count Accuracy	$\geq 90\%$	Measure inventory-count correctness
Inference Time	≤ 200 ms/image	Measure practical processing speed

These values are project targets, not guaranteed results. Final measurements will be reported after testing.

9. Risks and Mitigation

Occlusion: Restrict the main count to visible/front-facing products and document missed hidden items.

Similar packaging: Use broader product groups when variants cannot be reliably distinguished.

Dense shelves: Test different confidence thresholds and image resolutions.

Lighting / blur: Include varied lighting conditions and use preprocessing when appropriate.

False detections: Tune confidence thresholds and report false-positive/false-negative behavior.

10. Success Criteria

- The system accepts a shelf image and produces an annotated output.
- YOLOv8 detects visible products with measurable performance.
- The application calculates product counts from detection results.
- The inventory logic identifies low-stock or out-of-stock conditions using configurable thresholds.
- The project reports detection and counting metrics using a defined test set.
- The GitHub repository contains understandable code, documentation, example results, and project limitations.

11. Conclusion

SmartShelf Vision demonstrates how a computer vision model can be combined with application logic to address a practical retail inventory problem. The project follows the Tier 1 architecture of one model doing one primary job: YOLOv8 performs object detection, while Python and OpenCV handle counting, inventory evaluation, and visualization.

The proposed agent transforms a shelf photograph into a structured inventory result through the pipeline Detect → Count → Evaluate → Alert. This provides a clear and manageable foundation for an ITAI 1378 computer vision project while leaving room for future improvements such as custom model training, real-time camera input, historical inventory tracking, and dashboard integration.